Towards Convex Formulations in Calibration

Exploring Calibration and Registration over Lie Groups

1st Joseph Hobbs
josephrosshobbs@gmail.com

The great watershed in optimization is not between linearity and nonlinearity, but convexity and nonconvexity.

R. Tyrrell Rockafellar

***Abstract*—Calibration is a critical step in building safe and reliable autonomous systems. Cameras, LiDAR sensors, gyroscopes, inertial measurement units, and other devices give various information about the world around a system. Effective algorithms for calibration can quickly give the user correct parameters that characterize those devices—previously unknown positions, scales, focal lengths, distortion coefficients, or orientations in space. Many problems in calibration, however, suffer from various nonconvexities that make globally optimal estimates of these parameters difficult to determine correctly, certifiably, or in polynomial time. Clever convex reformulations of these problems can often bypass these difficulties and achieve correct and certifiable solutions quickly. Here, we study Similarity Registration with Known Correspondences (SRKC), a common problem in calibration and point cloud registration involving estimation over a Lie group under isotropic Gaussian noise. We develop a convex formulation and show how it extends to three variants of the problem: constrained SRKC (C-SRKC), quaternion SRKC (q-SRKC), and robust SRKC (R-SRKC). We discuss applications of the methods developed here to real perception and calibration systems, and to preserve pedagogical value, we choose to implement the accompanying code in Python and release it as open-source (josephrhobbs/towards-convex-calibration).**

***Index Terms*—calibration, point cloud registration, convex analysis, optimization, Lie groups.**

I. Calibration as Bayesian Estimation

A. Introduction

Calibration—an elusive yet essential problem in contemporary robotics—is defined as "the process of determining the parameters that define how a sensor captures data from the world" [1]. Though calibration systems deal with vast collections of multimodal data (imagery, LiDAR point clouds, gyroscopic measurements, etc.), the problem can always be formulated as a *Bayesian estimation problem* under a *generative model*. A highly applicable generative model for calibration—which captures many but not all problems in the field—is given by

$$y = f(x;\theta) + \varepsilon. \tag{1}$$

Here, I use y to denote sensor measurements, x to denote the state of the environment, θ to denote *sensor parameters*, and ε to denote additive measurement noise. (I would like to note that measurement noise is not always additive, e.g. in the case that $y \in \mathrm{SO}(3)$).

B. Measurement as a Bayesian Update

The process of *calibration* in this case involves solving for θ given a collection of values for (x_i, y_i). The Bayesian approach to calibration is rather simple in principle, albeit often quite complex in practice. All we do is assume that θ is a random variable distributed according to some prior density function $p_\Theta(\theta)$. In this case (but not all cases!) we assume that we know the state of the environment x and we are measuring y from our sensor. This gives us an *a posteriori* distribution for θ according to

$$p_\Theta(\theta|x,y) = \frac{1}{Z} p_\Theta(\theta) p_Y(y|\theta,x) \tag{2}$$

where $Z := \int p_Y(y|\theta',x)\,d\theta'$ is a normalization constant, which in practice is typically intractable. No matter. We will see Z disappear quite soon.

C. Maximum a Posteriori Estimation

The Bayesian update above gives a *distribution* over sensor parameters θ, but we really want an *estimator* of the sensor parameters. Therefore, we apply a *maximum a posteriori* (MAP) estimator to $p_\Theta(\theta|x,y)$ to obtain a final estimate of the calibration parameters.

$$\theta^\star = \arg\max_\theta p_\Theta(\theta|x,y) \tag{3}$$

There are other ways to estimate (e.g. maximum likelihood estimation, minimum mean squares estimation, etc.) and in the case of Gaussian noise ε they are all in fact equivalent. However, maximum a posteriori estimation is elegant because it lends itself well to formulating calibration as an optimization. We will use *negative log-likelihood* (NLL) to convert this into a minimization problem.

$$\theta^\star = \arg\min_\theta -\log p_\Theta(\theta|x,y) \tag{4}$$

Substituting our previous expression for the posterior and dropping the constant term, we get the following.

$$\theta^\star = \arg\min_\theta -\log(\frac{1}{Z}p_\Theta(\theta)p_Y(y|\theta,x)) \tag{5}$$
$$= \arg\min_\theta [\log Z - \log p_\Theta(\theta) - \log p_Y(y|\theta,x)] \tag{6}$$
$$= \arg\min_\theta [-\log p_\Theta(\theta) - \log p_Y(y|\theta,x)] \tag{7}$$

Our final step will be to use our generative model (1) to rewrite the evidence (the second term). We know that, because $y = f(x;\theta) + \varepsilon$, the only variability in y derives from the measurement noise ε under fixed x, θ. Formally,

$$p_Y(y|\theta,x) = p_\epsilon(y - f(x;\theta)) \tag{8}$$

and we write our final formulation for the calibration problem.

$$\boxed{\theta^\star = \arg\min_\theta [-\log p_\Theta(\theta) - \log p_\epsilon(y - f(x;\theta))]} \tag{9}$$

D. Similarity Registration under Known Correspondences

I address here the challenges of calibration—the challenges of solving the optimization (9). Throughout this document, I will use a particular problem which I will call *Similarity Registration under Known Correspondences* (SRKC) to demonstrate the relevant mathematical techniques. Given two point clouds, which I will call $x \in \mathbb{R}^{3\times N}$ and $y \in \mathbb{R}^{3\times N}$, the objective of SRKC is to estimate the similarity transform $S \in \mathrm{Sim}(3)$ that relates x to y *under known correspondences*. A similarity transform is the composition of a rigid (distance-preserving) transform over space $X \in \mathrm{SE}(3)$ composed with a global scale $s \in \mathbb{R}_+$. For a correspondence pair (x_i, y_i) this looks like

$$y_i = sRx_i + t + \varepsilon_i \tag{10}$$

where s is the global scale, $R \in \mathrm{SO}(3)$ is a proper rotation, $t \in \mathbb{R}^3$ is a (scaled) translation, and $\varepsilon \in \mathbb{R}^3$ is an additive noise. Here, we will assume that ε_i is isotropic Gaussian noise with known covariance $\sigma^2 I_3$. Finally, we take the priors as

$$s \sim \mathcal{N}(0, \sigma_1^2) \mid s \geq 0 \tag{11}$$
$$R \sim \mathrm{Unif}(\cdot) \mid R \in \mathrm{SO}(3) \tag{12}$$
$$t \sim \mathcal{N}(0, \sigma_2^2 I_3) \mid t \in \mathbb{R}^3 \tag{13}$$

and then take $\sigma_1^2, \sigma_2^2 \to \infty$ such that we assume to have no prior information whatsoever. This actually allows us to drop the prior term from the optimization objective and optimize only over the evidence term. Applying our formulation for calibration we developed earlier, the optimization relevant to SRKC is as follows.

$$s^\star, R^\star, t^\star = \arg\min_{s,R,t} \frac{1}{2\sigma^2} \sum_{i=1}^{N} \left[\|y_i - sRx_i - t\|_2^2\right] \tag{14}$$

I know that SRKC doesn't look like a typical calibration problem. It looks like a point registration problem—because it is a point registration problem. On the one hand, SRKC is definitely useful in calibration. Imagine I have a LiDAR sensor that outputs point clouds and I am calibrating it against an object with a known shape (like a 3D-printed Stanford bunny) [2], but the sensor outputs point locations in an arbitrary and unknown distance unit. SRKC would be incredibly useful in a "similarity iterative closest-point" (similarity ICP) calibration tool for this LiDAR sensor. On the other hand, solving SRKC is a wonderful exercise because it exposes so clearly many of the problems in calibration. It requires us to work with Lie groups and non-Euclidean manifolds, it has plenty of local minima to frustrate us, and yet with a bit of coaxing it quickly admits powerful solution techniques which give global optimality.

E. A Note about Solution Methods

I would like to note that this work does not discount the value of the Gauss-Newton method, the Levenberg-Marquardt method, and other "local" solvers, neither does it discount the value of probabilistic or exhaustive methods for robust calibration such as Random Sample Consensus (RANSAC) or Branch-and-Bound (BnB) algorithms. Although they lack theoretical guarantees, these solvers are often highly effective at calibration in production systems, and with some mild discomfort and hyperparameter tweaking on the user's part they can work extremely well in practice on both convex and nonconvex problems.

I have prepared this report and its associated code to highlight the benefits that convex optimization offers in many (but not all) cases. I encourage us both to remember, as we work through solution methods for SRKC, that convex optimization should be *another tool in our calibration toolbox*, rather than the "end-all-be-all" approach for every situation. Convex programming, despite its polynomial-time convergence, guarantee of global optimality, and highly expressive nature, can still be slower in practice, even if a convex form exists. But many times, it is an incredibly powerful tool that reveals efficient algorithms for certifiable solutions to problems in calibration.

II. Nonconvexities in Calibration

A. Nonconvexity from Bilinearity

The optimization (14) allows us to solve for the similarity transform $s^\star, R^\star, t^\star$ that relates two point clouds. We write the objective again below for convenience, dropping the irrelevant constant factor.

$$s^\star, R^\star, t^\star = \arg\min_{s,R,t} \sum_{i=1}^{N} \left[\|y_i - sRx_i - t\|_2^2\right] \tag{15}$$

The (squared) ℓ_2 norm is a convex function of its input, but the inner vector term $y_i - sRx_i - t$ is not convex jointly in s, R, t—*bilinearity* introduced by products of variables (such as sR) is nonconvex. The objective is a linear combination of convex, non-decreasing functions (ℓ_2 norms) of nonconvex expressions, so the entire objective here is nonconvex [3].

This is strongly representative of the challenges due to nonconvexity in optimization-based calibration—in general, any time a product of decision variables arises, we can expect the objective to be nonconvex (unless a simplification exists).

B. Nonconvexity from Rotations

The bilinearity is not the only source of nonconvexity in (14). Unfortunately for us, R must be a proper rotation ($R \in \mathrm{SO}(3)$) and the set of all proper rotations (the Lie group $\mathrm{SO}(3)$) is nonconvex. This nonconvexity is also highly representative of the type encountered in calibration. Estimating rotation is critical in nearly every calibration problem, and the set of all rotations is a nonconvex set. We will see the challenges that this presents in a bit—and we will also explore methods for coaxing the problem into a convex formulation for solution and formal certification. However, a final nonconvexity exists in some calibration problems as well, albeit a more nefarious and more subtle type.

C. Nonconvexity from Mixed-Integer Formulations

Nonconvexity also originates in calibration through *robust approaches* that have mixed-integer formulations. Often, part of the calibration process is determining which data are "inliers" and which data are "outliers". For N independent measurements, there are 2^N possible classifications. The "inlier"/"outlier" taxonomy can be written in terms of binary decision variables $\theta_i = \pm 1$ for each of $i = 1, 2, \cdots N$, such that $\theta_i = +1$ indicates that point i is an inlier and $\theta_i = -1$ indicates that point i is an outlier. This allows us to construct *mixed-integer programming* (MIP) formulations for robust calibration or registration systems. MIP is inherently nonconvex (as the feasible set is disconnected) and NP-hard due to its combinatorial nature.

III. Separability of Calibration Problems

Many problems in calibration requiring joint estimation can be "separated" into multiple independent problems—and SRKC is no exception! Though SRKC involves joint estimation of $s^\star, R^\star, t^\star$, we can decompose the problem into estimating each of these *sequentially* rather than all at once [4]. Because the objective

$$s^\star, R^\star, t^\star = \arg\min_{s,R,t} \sum_{i=1}^{N} \left[\|y_i - sRx_i - t\|_2^2 \right] \tag{16}$$

is convex, and because there are no constraints $s^\star, t^\star$, we can actually write the global scale $s^\star$ as a function of the global rotation $R^\star$, and the global translation $t^\star$ as a function of both the scale $s^\star$ and the rotation $R^\star$. I'll save you the time and skip the derivations.

$$s^\star = \frac{\sum_{i=1}^{N} y_i^\top R x_i - \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{N} y_j^\top R x_i}{\sum_{i=1}^{N} x_i^\top x_i - \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{N} x_j^\top x_i} \tag{17}$$

$$t^\star = \frac{1}{N}\sum_{i=1}^{N} \left[y_i - s^\star R^\star x_i \right] \tag{18}$$

Substituting these solutions back into the original objective and simplifying (through a great deal of algebra), we get an extraordinarily elegant formulation for R.

$$R^\star = \arg\max_{R \in \mathrm{SO}(3)} \mathrm{tr}(RH)^2 \tag{19}$$

$$\text{where } H := \sum_i (x_i - \bar{x})(y_i - \bar{y})^\top \tag{20}$$

and $\bar{x}, \bar{y}$ represent the centroids of point clouds x, y. Because x^2 is monotonically increasing in x, we can make one final change to our expression, and instead write

$$R^\star = \arg\max_{R \in \mathrm{SO}(3)} \mathrm{tr}(RH), \tag{21}$$

$$\text{where } H := \sum_i (x_i - \bar{x})(y_i - \bar{y})^\top. \tag{22}$$

This problem easily admits the solution to the *orthogonal Procrustes* problem for rotation estimation. Simply compute the singular value decomposition (SVD) of H, then write

$$H = U \Sigma V^\top \tag{23}$$

$$\therefore R^\star = V U^\top \tag{24}$$

and derive $s^\star, t^\star$ subsequently.[1] However, under various modifications, such as constrained SRKC or robust SRKC, this problem no longer admits the same SVD solution. Therefore it is important to develop a more generalizable way of thinking about SRKC. This brings us to the main focus of this document: exploiting *convexity* in calibration and registration.

IV. Convex Formulations for Calibration

We're now at the heart of this. We just said above that our constraint $R \in \mathrm{SO}(3)$ was nonconvex. So why are we thinking about convexity in calibration? Because *properly formulated, many optimizations can be treated as convex* even if the original problem was not. I emphasized earlier that a benefit of convexity is *certifiability*. This means that, if the Karush-Kuhn-Tucker conditions (KKT) are satisfied for an optimizer of the convex formulation, we have *mathematical proof* that we've found the *global optimizer* for our estimation problem [5]. It is a tremendous peace of mind to have *rigorous proof that we can't do any better*—if our solution is KKT, then there is no better solution elsewhere in the feasible set.

A. Relaxation of the Special Orthogonal Group

Our first example here will be to relax the constraint $R \in \mathrm{SO}(3)$. A necessary (but not sufficient)[2] condition for $R \in \mathrm{SO}(3)$ is $I_3 - R^\top R = 0$; in other words, the matrix columns

[1] Technically, this doesn't *quite* work because it has no way of enforcing $\det(R^\star) = 1$; it only enforces $\det(R^\star) = \pm 1$. If the determinant is negative, then it is easy to write $R^\star = V\,\mathrm{diag}(1, 1, -1)\,U^\top$.

[2] Again, we must enforce positivity of the determinant to make sure this is a *proper* rotation. And again, this is not difficult using a quick SVD check at the end.

must form an orthonormal basis for $\mathbb{R}^3$. Funnily enough, we can actually relax this constraint to

$$I_3 - R^\top R \succeq 0 \tag{25}$$

(in other words, $I_3 - R^\top R$ has no negative eigenvalues) and we typically recover globally optimal solutions in practice. This works because the *positive semidefinite cone* (the set of all matrices with no negative eigenvalues) is actually convex, whereas the set $\mathrm{SO}(3)$ is nonconvex, and *critically*, all proper rotations satisfy this new condition. This expression is actually the Schur complement of the larger expression

$$\begin{bmatrix} I_3 & R \\ R^\top & I_3 \end{bmatrix} \succeq 0 \tag{26}$$

and so, in practice, we use this constraint instead because it is linear in R. We now have a completely *convex* formulation by removing the constraint $R \in \mathrm{SO}(3)$ and replacing it with our new *linear matrix inequality* (LMI).

$$R^\star = \arg\max_R \mathrm{tr}(RH), \tag{27}$$

$$\text{where } H := \sum_i (x_i - \bar{x})(y_i - \bar{y})^\top \tag{28}$$

$$\begin{bmatrix} I_3 & R \\ R^\top & I_3 \end{bmatrix} \succeq 0 \tag{29}$$

This is easily solved using an "off-the-shelf" semidefinite programming (SDP) solver, such as CVX [7]. In practice, this will nearly always recover a proper rotation $R \in \mathrm{SO}(3)$, even though we did not explicitly impose that requirement. However, if the problem does not recover a proper rotation, then a simple projection onto the special orthogonal group will recover the closest valid rotation (in the least-squares sense).

$$R^\star = U\Sigma V^\top \tag{30}$$

$$R^\star \leftarrow U \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det UV^\top \end{bmatrix} V^\top \tag{31}$$

B. Constrained SRKC

A semidefinite programming formulation of "vanilla" SRKC is technically redundant—after all, the problem can be solved without much difficulty by first eliminating $s^\star, t^\star$ and then using SVD to determine the optimal rotation matrix $R^\star$. However, in the presence of additional constraints, this solution no longer works and our SDP formulation comes to the rescue. We call this the *constrained similarity registration under known correspondences* (C-SRKC) problem. The SVD approach to SRKC is no longer useful to us in the C-SRKC variant—we must solve the problem as a convex optimization. The optimization is written as

$$s^\star, R^\star, t^\star = \arg\min_{s,R,t} \sum_{i=1}^N \left[\|y_i - sRx_i - t\|_2^2 \right], \tag{32}$$

$$\text{s.t. additional contraints.} \tag{33}$$

This optimization is nonconvex because the term sR in the objective is bilinear and therefore, in general, nonconvex. Furthermore, the constraint $R \in \mathrm{SO}(3)$ limits the decision variable R to a nonconvex feasible set. Not all hope is lost, however. With another relaxation, we can solve for the optimal similarity transform using a convex optimization. First, we introduce the matrix $Q := sR$ to write

$$s^\star, Q^\star, t^\star = \arg\min_{s,Q,t} \sum_{i=1}^N \left[\|y_i - Qx_i - t\|_2^2 \right], \tag{34}$$

$$\text{subject to } n^\top (Qx_i + t) + b \geq 0, \tag{35}$$

$$s^2 I_3 - Q^\top Q = 0. \tag{36}$$

We have now eliminated one of the two nonconvexities, and we introduced the constraint $Q^\top Q = s^2 I_3$ as a necessary (but not sufficient) condition for $R \in \mathrm{SO}(3)$. Notice that, because s only appears as s^2 here, it is not necessary to constrain $s \geq 0$. We can relax this constraint to

$$s^2 I_3 - Q^\top Q \succeq 0 \tag{37}$$

and linearize it by introducing $\gamma := s^2$. This eliminates s completely, leaving us only with $\gamma I_3 - Q^\top Q \succeq 0$, which is equivalent to an LMI via the Schur complement "trick" I showed earlier, and therefore induces a convex feasible set. Next, we eliminate translation as a decision variable using the result

$$t^\star = \frac{1}{N} \sum_{i=1}^N [y_i - s^\star R^\star x_i] \tag{38}$$

from the previous section. We write $a_i := x - \bar{x}$ and $b_i := y_i - \bar{y}$ for the points in each point cloud, with a and b having a shared centroid at the origin, removing t as a decision variable. The final steps are to interpret this LMI as the Schur complement of a larger expression, allowing the optimization to admit a standard semidefinite programming form, and to write $\gamma \geq 0$ because $s^2 \geq 0$.

$$\gamma^\star, Q^\star = \arg\min_{\gamma,Q} \sum_{i=1}^N \left[\|b_i - Qa_i\|_2^2 \right], \tag{39}$$

$$\text{subject to } \begin{bmatrix} \gamma I_3 & Q \\ Q^\top & I_3 \end{bmatrix} \succeq 0 \tag{40}$$

$$\gamma \geq 0 \tag{41}$$

$$\text{additional constraints.} \tag{42}$$

Again, this is efficiently solved by an "off-the-shelf" semidefinite programming solver [7], and in practice this relaxation will often be tight. We may now write any additional constraints we choose in (42), including global scale constraints or constraints on translation. Notice that, conveniently, we can write the translation as affine in the decision variable Q, which makes constraints on t easy to write.

$$t = \frac{1}{N}\sum_{i=1}^{N}[y_i - Qx_i] \tag{43}$$

Obviously, we've lost sight of the *actual* variables we care about, $s^\star, R^\star, t^\star$. But it's not difficult to recover those using the following relations.

$$s^\star = \sqrt{\gamma^\star} \tag{44}$$
$$Q^\star = U\Sigma V^\top \tag{45}$$
$$R^\star = U\begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & \det UV^\top\end{bmatrix}V^\top \tag{46}$$
$$t^\star = \frac{1}{N}\sum_{i=1}^{N}[y_i - s^\star R^\star x_i] \tag{47}$$

C. Quaternion SRKC

Our semidefinite programming solution to SRKC is tolerant to decent amounts of noise. However, an SDP solver is a relatively specialized piece of software that is not always available on every platform. Moreover, many of them do not have permissive licenses (e.g. commercial users must purchase a license), which further limits the practicality of SDP for SRKC. In the case that an SDP solver is impractical, we can actually reformulate the problem in terms of *unit quaternions* and solve it to global optimality using only simple linear-algebraic operations. This may also be preferable to the SVD method for vanilla SRKC under certain circumstances. We will call this *quaternion similarity registration under known correspondences* (q-SRKC). Again I will skip the messy derivation, but it turns out that if you replace the matrix R with the unit quaternion q, the optimal rotation is

$$q^\star = \arg\max_{q^\top q=1} q^\top F q, \tag{48}$$
$$\text{where } F := \begin{bmatrix}\sigma & z^\top\\ z & S - \sigma I_3\end{bmatrix}, \tag{49}$$
$$\sigma := \operatorname{tr}(H), \tag{50}$$
$$S := H + H^\top, \tag{51}$$
$$z := \begin{bmatrix}H_{23} - H_{32}\\ H_{31} - H_{13}\\ H_{12} - H_{21}\end{bmatrix} \tag{52}$$

and H is the matrix computed in (22). The constraint $q^\top q = 1$ makes this optimization nonconvex. However, it is not difficult to recognize that the objective is maximized when q is (anti)parallel to the eigenvector of F with maximum associated eigenvalue. The *power iteration* algorithm is highly efficient for computing the maximum eigenvector associated to F. Simply select q_0 to be any quaternion you wish, then iteratively compute

$$q_{k+1} \leftarrow Fq_k / \left\|Fq_k\right\|_2 \tag{53}$$

until the algorithm converges, and q_k is guaranteed to be the eigenvector of F with maximum eigenvalue (up to numerical precision). Because this algorithm returns unit quaternions by construction, it is not necessary to project its results onto $\mathrm{SO}(3)$, unlike the methods for SRKC and C-SRKC. This algorithm is by no means novel—it is, in fact, Davenport's q-method used by attitude determination systems aboard spacecraft [8].

D. Robust SRKC

The introduction of quaternions is not only useful for solving SRKC in the absence of an SDP solver—it is also useful for solving the SRKC problem in the presence of *outlier correspondences*. Imagine that, with probability $1 - \eta$ (where η is the "inlier fraction"), the point pair (x_i, y_i) is labeled as a correspondence pair despite *not* being related by the generative model

$$y_i = sRx_i + t + \varepsilon_i. \tag{54}$$

This is considered an *outlier correspondence*. In the presence of outlier correspondences, we can develop an alterative method for SRKC which we will refer to as *robust similarity registration under known correspondences* (R-SRKC). Suppose the maximum permissible ℓ_2 norm of noise is $\bar{c}$, and if the noise under the generative model exceeds $\bar{c}$ for a given correspondence pair, we will label it an outlier correspondence. Then the optimal quaternion $q^\star$ is given by

$$q^\star = \arg\min_{q^\top q=1}\sum_{i=1}^{N}\min\left(\left\|y_i - sq\circ x_i\circ q^{-1} - t\right\|_2^2, \bar{c}^2\right) \tag{55}$$

under known global scale s and known translation t. This is a *horribly* nonconvex problem. The inner $\min(\cdot,\cdot)$ operation induces a combinatorial explosion in the feasible set. Under a specific reformulation, the feasible set is

$$\{-1, 1\}^N \times \mathrm{Sim}(3)$$

which is nonconvex in the worst way: the feasible set is split into 2^N possible disconnected sets! There in fact exists a polynomial-time convex formulation for this, which enables *certifiable global optimality* despite not knowing which correspondences are inliers and which are outliers. I've omitted the full mathematical derivation here, as it is *quite* involved, but I encourage you to check out [6] if you're curious—this is a recently published result in geometric perception. Due to its extreme complexity, I have omitted an implementation of R-SRKC from the accompanying source code.

I'd like to note that, in practice, robust SRKC can be achieved with not much difficulty using graduated nonconvexity (GNC). But I wanted to take the time to emphasize that you can in fact solve this to *certifiable* global optimality by finding the right convex formulation.

V. Discussion

We've now seen four ways of approaching the problem of Similarity Registration under Known Correspondences.

1) A *vanilla* SRKC approach, using either SVD or convex optimization to solve for the optimal rotation, followed by determination of the optimal scale and translation;
2) A *constrained* SRKC approach which extends the vanilla SRKC convex formulation in the presence of constraints, such as minimum or maximum global scales;
3) A *quaternion* SRKC approach which solves the unconstrained problem in the absence of a dedicated semidefinite programming or singular value decomposition solver; and
4) A *robust* SRKC approach which simultaneously solves the registration problem and classifies correspondences as "inlier" or "outlier".

All of these methods but q-SRKC involved a convex optimization, and all of them enabled the problem to be solved to *global optimality* efficiently. This highlights the expressivity of convex optimization as a language for expressing problems in calibration.

Of course, as I mentioned above, I have no intention of discounting the benefits of Gauss-Newton, Levenberg-Marquardt, RANSAC, BnB, or other algorithms frequently used in calibration! I selected SRKC as a problem of study and formulated three of its variants as a convex optimization because convex programming has two qualities I greatly value. Firstly, convex optimizations *do not require a good initial guess* to get a good final answer—because the problem is convex, there is one global optimum and your iterates will converge towards it by the properties of the algorithm your solver uses. Secondly, convex optimizations *always achieve the global optimum* and never fall into local minima—iterates never "get stuck" and stall out in a poor solution.

These are both strongly desirable properties in calibration, because users are more likely to trust solvers that always output good answers without the need for manual tuning or "fiddling with hyperparameters", and accurate calibration in the presence of uncertainty means safer, more reliable autonomous systems.

Acknowledgements

I am deeply grateful to you, dear reader, for your time and attention.

Generative AI Statement

I hold a deep respect for the inspired applied mathematicians, engineers, and researchers working to build the future of generative models for language, code, and images ("generative AI"). However, at the same time, I hold myself to the highest standard of accountability and accuracy in my work. I also recognize that research, development, and writing are deeply human processes that deserve human attention. Therefore, it is my guarantee that neither this document nor any associated content (including figures and source code) were created or copyedited by generative AI.

References

[1] Tangram Vision. "Welcome to MetriCal." *MetriCal Documentation*, Tangram Vision, https://docs.tangramvision.com/metrical/14.1/intro/

[2] Stanford Computer Graphics Laboratory. "The Stanford 3D Scanning Repository." Stanford Computer Graphics Laboratory, Stanford University. https://graphics.stanford.edu/data/3Dscanrep/

[3] S. Boyd, S. Diamond, and J. Park. "Constructive Convex Analysis and Disciplined Convex Programming." Stanford University, https://web.stanford.edu/ boyd/papers/pdf/dcp.pdfs

[4] H. Yang, J. Shi, and L. Carlone. "TEASER: Fast and Certifiable Point Cloud Registration." Laboratory for Information and Decision Systems, Massachusetts Institute of Technology. https://arxiv.org/pdf/2001.07715

[5] R. Tibshirani. "Karush-Kuhn-Tucker Conditions." Carnegie Mellon University, https://www.stat.cmu.edu/ ryantibs/convexopt/lectures/kkt.pdf

[6] H. Yang and L. Carlone. "A Quaternion-based Certifiably Optimal Solution to the Wahba Problem with Outliners." Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, https://arxiv.org/pdf/1905.12536

[7] CVX Research. "Software for Disciplined Convex Programming." CVX Research, https://cvxr.com/

[8] M. Garcia. "Davenport's q-Method." *Attitude and Heading Reference Systems*, https://ahrs.readthedocs.io/en/latest/filters/davenport.html